D-HNSW: Deterministic Hierarchical Navigable Small World Graphs for Verifiable Nearest Neighbor Search

Anonymous Authors Under Review

Abstract—Approximate nearest neighbor (ANN) search is a cornerstone of modern data-intensive applications, from recommendation systems to retrieval-augmented generation. The Hierarchical Navigable Small World (HNSW) algorithm has emerged as the dominant index structure due to its superior recall-throughput tradeoff. However, HNSW relies on IEEE 754 floating-point arithmetic, which introduces platform-dependent nondeterminism through variations in instruction reordering, fused multiply-add availability, and SIMD lane widths. This nondeterminism renders search results unverifiable—a critical limitation for blockchain-based systems, decentralized AI, and regulatory-compliant deployments where independent parties must reproduce identical results.

We present D-HNSW (Deterministic HNSW), a modified HNSW algorithm that guarantees bit-exact reproducibility across all platforms by replacing floating-point distance computations with 64-bit fixed-point integer arithmetic. Our fixed-point representation uses a Q32.32 format with saturating operations and achieves distance computation errors below 10^{-8} relative to IEEE 754 double precision. To mitigate the inherent performance overhead of integer arithmetic, we introduce a suite of optimizations including SIMD-accelerated fixed-point operations, two-phase distance computation with early termination, and partial distance pruning.

Extensive experiments on six benchmark datasets (SIFT-128, GloVe-100, Fashion-MNIST-784, GIST-960, Deep-96, and Random-128) demonstrate that D-HNSW achieves **identical** recall to standard HNSW while maintaining 57.1% of its throughput at the operating point of ef $= 128$ on SIFT-1M. Ablation studies show that our optimization pipeline reduces the naïve fixed-point overhead from $2.10\times$ to $1.75\times$. Determinism is verified through SHA-256 hash comparison across five independent executions, confirming bit-exact result reproducibility with zero hash collisions. The system scales to one million vectors while maintaining consistent overhead ratios, demonstrating practical viability for production deployment in trust-critical environments.

Index Terms—Approximate nearest neighbor search, deterministic computation, fixed-point arithmetic, HNSW, blockchain, verifiable AI, vector databases

1 Introduction

Approximate nearest neighbor (ANN) search has become an indispensable primitive in modern computing, underpinning applications from large-scale information retrieval [**?**] and recommendation systems to the retrieval-augmented generation (RAG) pipelines that power contemporary large language models. As vector databases grow to billions of entries [**?**], the demand for algorithms that balance search quality (recall) with computational efficiency (throughput) has intensified. Among the diverse family of ANN algorithms—including tree-based methods [**?**], locality-sensitive hashing [**?**], product quantization [**?**], and graph-based approaches [**?**]—the Hierarchical Navigable Small World (HNSW) algorithm [**?**] has established itself as the de facto standard, consistently achieving state-of-the-art recall-throughput tradeoffs across major benchmarks [**?**] and serving as the backbone of production systems such as Faiss [**?**], Qdrant, Weaviate, and Milvus.

Despite its effectiveness, HNSW harbors a fundamental limitation that has received insufficient attention: *nondeterminism*. Standard HNSW implementations rely on IEEE 754 floating-point arithmetic for distance computations, which, while providing excellent numerical range and precision, does not guarantee identical results across different hardware platforms, compiler optimization levels, or instruction set architectures [**?**]. Floating-point nondeterminism arises from multiple sources: (1) the availability and use of fused multiply-add (FMA) instructions, which compute $a \times b + c$ with a single rounding step rather than two; (2) compiler-driven instruction reordering that exploits algebraic properties not preserved under finite-precision arithmetic; (3) variations in SIMD lane widths (SSE, AVX2, AVX-512, NEON) that change the order of partial-sum accumulation; and (4) transcendental function implementations that differ across math libraries. These seemingly minor discrepancies can propagate through the greedy graph traversal of HNSW, where a single tie-breaking difference in distance comparison may redirect the search path, potentially yielding entirely different result sets.

For many traditional applications, such nondeterminism is inconsequential—a search engine returning slightly different results across servers is perfectly acceptable. However, a growing class of applications *requires* deterministic, verifiable computation:

- **Blockchain and smart contracts.** Decentralized systems require all validator nodes to reach identical state transitions from identical inputs [**?**], [**?**]. A vector similarity search embedded in a smart contract must produce bit-exact results regardless of the validator's hardware, or consensus will fail. Cardano's EUTXO model explicitly mandates deterministic transaction evaluation [**?**], and integer arithmetic is widely recognized as essential for

blockchain consensus [?].

- **Verifiable and auditable AI.** As AI systems are deployed in regulated domains—healthcare diagnostics, financial credit scoring, autonomous vehicles—regulators increasingly demand that inference results be reproducible and auditable [?]. Floating-point nondeterminism undermines this requirement, as demonstrated by recent work on nondeterminism-aware verification [?].
- **Decentralized vector databases.** Emerging Web3 architectures for similarity search [?], [?] and AI-powered data trading [?] require that query results be independently verifiable. ANNProof [?] addresses verification at the proof level but does not eliminate the underlying nondeterminism in distance computation.
- **Deterministic AI infrastructure.** Projects such as Valori [?] and EigenAI [?] are building deterministic substrates for AI computation, recognizing that reproducibility is a prerequisite for trust in distributed AI systems.

In this work, we propose D-HNSW, a variant of the HNSW algorithm designed to ensure bit-exact result consistency across diverse hardware platforms. The core idea is to employ 64-bit fixed-point integer arithmetic in place of IEEE 754 floating-point distance computations, thereby producing identical results on any processor that supports standard integer operations—which encompasses virtually all modern hardware. Our main contributions are as follows:

1) **Fixed-point distance computation framework.** We design a Q32.32 fixed-point arithmetic system with saturating operations, supporting squared Euclidean distance, Euclidean distance (via Newton–Raphson integer square root), and cosine similarity. We provide formal error bounds showing relative errors below 10^{-8} compared to IEEE 754 double precision (Section 4), grounded in the Determinism Trilemma framework (Section 3.4).
2) **Performance optimization suite.** We introduce five complementary optimizations—SIMD-accelerated fixed-point operations, two-phase distance computation, neighbor reordering, early termination, and partial distance pruning—that collectively reduce the naïve fixed-point overhead from $2.10\times$ to $1.75\times$ (Section 5).
3) **Deterministic graph construction.** We replace all sources of nondeterminism in the HNSW index construction process, including the random level generator, with a deterministic alternative based on a seeded Keccak-256 permutation [?], ensuring that both index building and querying are fully reproducible (Section 6).
4) **Comprehensive empirical evaluation.** We evaluate D-HNSW on six diverse benchmark datasets spanning 96 to 960 dimensions, demonstrating identical recall, verified determinism through SHA-256 hashing, and practical throughput of 57.1% of standard HNSW across all datasets (Section 7).

Regarding the organization of this paper, Section 2 reviews prior work on ANN search, deterministic computation, and blockchain applications. Section 3 then supplies the necessary background on the HNSW algorithm and the problem of floating-point nondeterminism. The fixed-point arithmetic framework underlying D-HNSW and the associated performance optimization strategies are analyzed in Section 4 and Section 5, respectively. Section 6 addresses deterministic graph construction, after which Section 7 reports our experimental findings. Finally, Section 8 examines implications and limitations, and Section 9 concludes.

2 Related Work

2.1 Approximate Nearest Neighbor Search

The ANN problem has been studied extensively across multiple algorithmic paradigms. **Tree-based methods**, originating with KD-trees [?], partition the space hierarchically but suffer from the curse of dimensionality, degrading to linear scan beyond approximately 20 dimensions. **Hashing-based methods**, notably locality-sensitive hashing (LSH) [?], provide sublinear query time with theoretical guarantees but require substantial memory overhead for high recall. **Quantization-based methods**, particularly product quantization (PQ) [?] and its variants [?], achieve excellent compression ratios but introduce quantization error that limits recall at high precision operating points.

Graph-based methods have emerged as the dominant paradigm for high-recall ANN search. The Navigable Small World (NSW) algorithm [?] constructs a proximity graph with long-range links that enable logarithmic-time greedy routing. HNSW [?] extends NSW with a hierarchical structure inspired by skip lists, where upper layers contain progressively sparser subsets of nodes, enabling coarse-to-fine navigation. This design achieves $O(\log n)$ expected search complexity while maintaining high recall, and has been validated at billion scale [?]. DiskANN [?] adapts graph-based search for disk-resident indices, while BANG [?] leverages GPU parallelism. HARMONY [?] addresses distributed deployment. Recent work by Teofili and Lin [?] introduces patience-based early termination for HNSW traversal, a strategy complementary to our approach.

Despite the maturity of ANN algorithms, none of the above works address the determinism of distance computation. All assume IEEE 754 floating-point arithmetic and implicitly accept platform-dependent variation in results.

2.2 Deterministic Computation and Blockchain

The requirement for deterministic execution is well-established in blockchain systems. Ethereum [?] achieves consensus by requiring all nodes to execute identical EVM bytecode and arrive at the same state. Cardano's EUTXO model goes further by mandating deterministic transaction evaluation, where transaction fees and outcomes are predictable before submission [?]. The Decenomy project [?] explicitly identifies integer arithmetic as essential for blockchain consensus, noting that floating-point operations can produce different results across validator nodes.

Lai *et al.* [?] study the intersection of private blockchains and deterministic databases, demonstrating that database operations must be carefully constrained to maintain consensus. Olivieri *et al.* [?] present GoLiSA, a static analysis tool for ensuring determinism in blockchain software, highlighting the practical difficulty of eliminating nondeterministic behavior from complex software systems.

In the AI domain, Yao *et al.* [?] address nondeterminism in floating-point neural network verification, proposing optimistic verification strategies that account for platform-dependent variation. EigenAI [?] and Valori [?] represent

emerging efforts to build deterministic AI infrastructure, recognizing that verifiable computation requires eliminating all sources of nondeterminism, including floating-point arithmetic.

2.3 Vector Search on Blockchain

Several recent works have explored the intersection of vector similarity search and blockchain technology. ANNProof [**?**] builds a verifiable outsourced ANN search system on blockchain, using cryptographic proofs to verify that search results are correct. However, ANNProof operates at the verification layer and does not address the underlying nondeterminism of distance computation—if two nodes compute different floating-point distances, the proof system cannot reconcile the discrepancy.

Keizer *et al.* [**?**] propose Ditto, a decentralized similarity search system for Web3 services that distributes the index across multiple nodes. Lee and Chen [**?**] implement decentralized face identification using HNSW on blockchain. Zhang [**?**] introduces NMFT, a data trading protocol that combines NFTs with AI-powered Merkle feature trees for copyright verification. All of these systems would benefit from deterministic distance computation to ensure consensus across distributed nodes.

Positioning of D-HNSW. Our work is complementary to and distinct from the above efforts. While ANNProof provides verification *after* computation, D-HNSW ensures determinism *during* computation. While Ditto and DecentFace deploy HNSW on blockchain without addressing floating-point nondeterminism, D-HNSW provides the deterministic foundation that makes such deployments correct by construction. To our knowledge, D-HNSW is the first work to systematically eliminate floating-point nondeterminism from graph-based ANN search while maintaining practical performance.

3 Background

3.1 HNSW Algorithm

The HNSW algorithm [**?**] constructs a multi-layer proximity graph where each layer $\ell \in \{0, 1, \ldots, L\}$ contains a subset of the dataset vectors. Layer 0 contains all n vectors, while higher layers contain exponentially fewer nodes, with the probability of a vector appearing at layer ℓ given by $p(\ell) = e^{-\ell/m_L}$, where m_L is a normalization factor.

Index construction. Each vector v is inserted by first determining its maximum layer ℓ_v via random sampling. Starting from the entry point at the top layer, a greedy search descends through layers $L, L-1, \ldots, \ell_v+1$, finding the nearest neighbor at each layer. From layer ℓ_v down to layer 0, the algorithm performs a beam search with width `efConstruction` to identify the M nearest neighbors, which become the edges of v in the graph. A heuristic pruning step ensures that edges are diverse, preferring neighbors that are not too close to each other.

Search. Given a query q, the search begins at the entry point on the top layer and greedily descends to layer 1. At layer 0, a beam search with width `ef` (the search parameter) explores the graph, maintaining a priority queue of candidates and a result set of the `ef` nearest vectors found so far. The search terminates when no candidate in the queue is closer to q than the farthest element in the result set. The top-k results are extracted from the final result set.

Distance computation. The core operation in both construction and search is distance computation between vectors. For d-dimensional vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, the squared Euclidean distance is:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{d} (x_i - y_i)^2 \tag{1}$$

This computation dominates the runtime of HNSW, accounting for over 90% of total execution time in typical workloads.

3.2 Floating-Point Nondeterminism

IEEE 754 floating-point arithmetic [**?**] is nondeterministic across platforms for several reasons:

Fused multiply-add (FMA). The FMA instruction computes $a \times b + c$ with a single rounding, while the separate multiply-then-add sequence rounds twice. The difference is at most one unit in the last place (ULP), but this suffices to change comparison outcomes. FMA availability varies: x86 processors gained FMA3 with Haswell (2013), while ARM NEON has always included it.

Instruction reordering. Compilers may reorder floating-point additions to improve instruction-level parallelism. Since floating-point addition is not associative—$(a+b)+c \neq a+(b+c)$ in general—different summation orders produce different results. The accumulation order in Equation (1) is particularly sensitive, as it involves d additions of squared differences.

SIMD vectorization. Different SIMD widths (128-bit SSE, 256-bit AVX2, 512-bit AVX-512) process different numbers of elements per instruction, leading to different partial-sum trees and thus different final results. A distance computation using AVX-512 (8 floats per lane) will generally differ from one using SSE (4 floats per lane).

Propagation through graph traversal. In HNSW, a distance difference of even one ULP can change the ordering of candidates in the priority queue. Since the greedy search follows the nearest candidate at each step, a single reordering can redirect the entire search path, potentially visiting completely different regions of the graph and returning different result sets. This amplification effect means that platform-dependent floating-point variation is not merely a numerical nuisance but a correctness concern for applications requiring deterministic results.

3.3 Fixed-Point Arithmetic

Fixed-point arithmetic represents real numbers as scaled integers, with a fixed number of bits allocated to the integer and fractional parts. A Q$m.f$ format uses m bits for the integer part and f bits for the fractional part, representing values in the range $[-2^{m-1}, 2^{m-1} - 2^{-f}]$ with resolution 2^{-f}.

The key advantage of fixed-point arithmetic for determinism is that all operations reduce to integer arithmetic (addition, subtraction, multiplication, and bit shifts), which is defined to produce identical results on all platforms

conforming to the C/Rust language standards. There is no platform-dependent rounding mode, no FMA ambiguity, and no SIMD-width-dependent accumulation order (when the summation order is fixed in software).

The primary disadvantage is reduced dynamic range compared to floating-point. A 64-bit fixed-point Q32.32 format provides approximately 9.6 decimal digits of precision with a range of $\pm 2^{31} \approx \pm 2.1 \times 10^9$, compared to IEEE 754 double precision's approximately 15.9 decimal digits with a range of $\pm 1.8 \times 10^{308}$. For ANN search, where vector components are typically normalized or bounded, this range is sufficient, as we demonstrate empirically.

3.4 The Determinism Trilemma

The integration of ANN search into blockchain architectures requires addressing three independent sources of state divergence. We formalize this as the *Determinism Trilemma*: a consensus-safe ANN index must simultaneously satisfy three constraints.

Let $\Phi : (X, R) \to G$ denote the graph construction function, where X is an ordered sequence of vector insertions and R is a source of entropy. An ANN implementation is *consensus-safe* if all honest validator nodes v_1, v_2 produce a bit-identical graph serialization: $\Phi_{v_1}(X, R) \equiv \Phi_{v_2}(X, R)$.

Constraint 1: Cryptographic seeding. The system must derive layer assignment entropy from consensus-agreed artifacts. D-HNSW defines the seed as $\text{seed} = \mathcal{H}(\text{TxHash} \oplus \text{BlockHash})$, where $\mathcal{H}$ is Keccak-256 [**?**]. The XOR mitigates grinding attacks, as the block hash remains unpredictable at transaction broadcast time.

Constraint 2: Fixed-point arithmetic. All distance computations must bypass floating-point ALUs. D-HNSW employs I64F32 representation (32 integer bits, 32 fractional bits), providing $65{,}536\times$ greater precision than Q16.16 formats used in prior systems [**?**]. The maximum distortion for D-dimensional normalized embeddings is bounded by $\mathcal{E} \leq D \cdot 2^{-31} \cdot \max_i |x_i - y_i| + D \cdot 2^{-62}$, which evaluates to $\sim 7.15 \times 10^{-7}$ for 768-dimensional LLM embeddings—insufficient to cause routing divergence (see Section A for the full proof).

Constraint 3: Canonical ordering. Vector insertions must be sequenced deterministically. D-HNSW serializes all insertions by their transaction index within the finalized blockchain block, eliminating race conditions from concurrent insertion.

An HNSW implementation satisfying all three constraints is provably consensus-safe: given identical inputs (X, seed), Constraint 1 ensures identical layer assignments, Constraint 2 ensures identical distance comparisons, and Constraint 3 ensures identical insertion order. By induction over $|X|$ insertions, the resulting graph is bit-identical across all validators.

4 D-HNSW: Fixed-Point Distance Framework

This section presents the core technical contribution of D-HNSW: a fixed-point arithmetic framework that replaces all floating-point distance computations in HNSW with deterministic integer operations.

4.1 Q32.32 Fixed-Point Representation

We adopt a Q32.32 fixed-point format, where each value is stored as a 64-bit signed integer with 32 bits for the integer part and 32 bits for the fractional part. The mapping between a real value r and its fixed-point representation $\hat{r}$ is:

$$\hat{r} = \lfloor r \times 2^{32} + 0.5 \rfloor \tag{2}$$

and the inverse mapping is $r \approx \hat{r}/2^{32}$. The representation error from quantization is bounded by:

$$|r - \hat{r}/2^{32}| \leq 2^{-33} \approx 1.16 \times 10^{-10} \tag{3}$$

Saturating operations. To prevent integer overflow from silently corrupting results, we implement all arithmetic operations with saturation semantics:

$$\text{sat\_add}(a, b) = \text{clamp}(a + b, -2^{63}, 2^{63}-1) \tag{4}$$

$$\text{sat\_sub}(a, b) = \text{clamp}(a - b, -2^{63}, 2^{63}-1) \tag{5}$$

$$\text{sat\_mul}(a, b) = \text{clamp}((a \times b) \gg 32, -2^{63}, 2^{63}-1) \tag{6}$$

where $\gg$ denotes arithmetic right shift. The multiplication uses 128-bit intermediate results (available on all 64-bit platforms) to avoid precision loss before the shift.

4.2 Vector Representation

Each d-dimensional vector $\mathbf{x} = (x_1, \ldots, x_d)$ is converted to a fixed-point vector $\hat{\mathbf{x}} = (\hat{x}_1, \ldots, \hat{x}_d)$ using Equation (2). The conversion is performed once during index construction and stored persistently. The `FixedPointVector<D>` structure in our Rust implementation stores the d components as an array of `i64` values with compile-time dimension checking via const generics.

4.3 Distance Metrics

Squared Euclidean distance. The squared L_2 distance between fixed-point vectors $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ is computed as:

$$\hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}}) = \sum_{i=1}^{d} \text{sat\_mul}(\hat{x}_i - \hat{y}_i,\ \hat{x}_i - \hat{y}_i) \tag{7}$$

where the subtraction and accumulation use saturating operations. The summation order is fixed (index 1 through d), eliminating the associativity-dependent variation of floating-point accumulation.

Euclidean distance. When the actual (non-squared) Euclidean distance is required, we compute an integer square root using Newton–Raphson iteration [**?**]:

$$g_{k+1} = \frac{1}{2}\left(g_k + \frac{S}{g_k}\right) \tag{8}$$

where $S = \hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}})$ is the squared distance and g_0 is an initial estimate obtained from the position of the highest set bit. We perform 20 iterations, which guarantees convergence to the exact integer square root for all 64-bit inputs. All operations in Equation (8) use integer division (truncating), which is deterministic.

Cosine similarity. For cosine similarity, we compute:

$$\cos(\hat{\mathbf{x}}, \hat{\mathbf{y}}) = \frac{N(\hat{\mathbf{x}}, \hat{\mathbf{y}})}{D(\hat{\mathbf{x}}) \cdot D(\hat{\mathbf{y}})} \tag{9}$$

where the numerator and denominators are defined as:

$$N(\hat{\mathbf{x}}, \hat{\mathbf{y}}) = \sum_{i=1}^{d} \text{sat\_mul}(\hat{x}_i, \hat{y}_i)$$
$$D(\hat{\mathbf{z}}) = \text{sqrt}\Big(\sum_{i=1}^{d} \text{sat\_mul}(\hat{z}_i, \hat{z}_i)\Big) \tag{10}$$

where sqrt denotes the integer square root from Equation (8). The division uses integer division with appropriate scaling to maintain precision.

4.4 Error Analysis

Theorem 1 (Distance computation error bound)**.** *Let $\mathbf{x}, \mathbf{y} \in [-B, B]^d$ be vectors with components bounded by B, and let $D(\mathbf{x}, \mathbf{y})$ and $\hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}})/2^{32}$ denote the true and fixed-point squared Euclidean distances, respectively. Then:*

$$\left| \frac{D(\mathbf{x}, \mathbf{y}) - \hat{D}(\hat{\mathbf{x}}, \hat{\mathbf{y}})/2^{32}}{D(\mathbf{x}, \mathbf{y})} \right| \leq \frac{d \cdot (4B + 1) \cdot 2^{-32}}{D(\mathbf{x}, \mathbf{y})} \tag{11}$$

provided no saturation occurs during computation.

Proof sketch. Each component difference $x_i - y_i$ incurs a quantization error of at most 2^{-32}. The squaring operation $(\hat{x}_i - \hat{y}_i)^2$ then has error bounded by $(2|x_i - y_i| + 2^{-32}) \cdot 2^{-32} \leq (4B + 1) \cdot 2^{-32}$ per component. Summing over d components and dividing by the true distance yields the bound. □

For typical ANN workloads with normalized vectors ($B \leq 1$) in moderate dimensions ($d \leq 1000$), this bound evaluates to relative errors well below 10^{-8}, as confirmed by our experiments (Section 7.4). The full suite of formal proofs—including bounds for Euclidean distance (Theorem 4), inner product (Theorem 5), edge reversal probability (Theorem 6), distance ordering preservation (Theorem 7), recall preservation (Theorem 8), overflow safety (Theorem 9), and cross-platform determinism (Theorem 10)—is provided in Section A. Theorem 3 in the appendix gives the complete proof of the squared distance bound stated above.

4.5 Determinism Guarantee

Theorem 2 (Bit-exact reproducibility)**.** *Given identical input vectors and identical algorithm parameters,* D-HNSW *produces bit-identical search results on any platform supporting 64-bit integer arithmetic, regardless of processor architecture, compiler, optimization level, or operating system.*

Proof sketch. The proof follows from three observations: (1) All distance computations use only integer addition, subtraction, multiplication, division, and bit shifts, which are defined to produce identical results by the C11/Rust language standards on all conforming implementations. (2) The summation order in distance computation is fixed by the source code (index 1 through d), eliminating associativity-dependent variation. (3) The graph construction uses a deterministic pseudo-random number generator (Keccak-256 permutation with a fixed seed), and all tie-breaking in the search algorithm uses deterministic comparison functions. Therefore, the entire computation is a deterministic function of its inputs. □

5 Performance Optimizations

The naïve replacement of floating-point with fixed-point arithmetic incurs a significant performance overhead (approximately $2.10\times$ on SIFT-1M), primarily because: (1) 64-bit integer multiplication is slower than 32-bit floating-point multiplication on modern processors with wide FP pipelines; (2) the saturating arithmetic requires additional comparison and clamping operations; and (3) the fixed summation order prevents certain compiler optimizations. We introduce five complementary optimizations that collectively reduce this overhead to approximately $1.75\times$.

5.1 SIMD-Accelerated Fixed-Point Operations

Modern processors provide SIMD instructions for integer operations that can be leveraged for fixed-point distance computation. We implement vectorized distance computation using platform-specific intrinsics:

- **x86-64 (AVX2):** We use `_mm256_sub_epi64` for component-wise subtraction and a custom 64-bit multiply-accumulate sequence using `_mm256_mul_epu32` with appropriate shuffling to compute the full 128-bit product. The accumulation processes 4 components per iteration.
- **ARM (NEON):** We use `vsubq_s64` for subtraction and `vmull_s32` / `vmlal_s32` for the multiply-accumulate, processing 2 components per iteration.

Critically, unlike floating-point SIMD where different lane widths produce different results due to different accumulation trees, our integer SIMD implementation produces *identical* results regardless of SIMD width, because integer addition is truly associative and commutative. The SIMD implementation is purely a performance optimization that does not affect the determinism guarantee.

This optimization reduces the overhead from $2.10\times$ to $1.836\times$ on SIFT-1M, a 14.4% improvement.

5.2 Two-Phase Distance Computation

We observe that in HNSW search, the majority of distance computations result in candidates that are farther than the current search radius and are immediately discarded. We exploit this by splitting the distance computation into two phases:

Phase 1 (Coarse filter): Compute the distance using only the first $d/4$ dimensions. If this partial distance already exceeds the current search radius, the candidate is rejected without computing the remaining dimensions.

Phase 2 (Full computation): If the partial distance is within the search radius, compute the full d-dimensional distance.

Since the partial distance is a lower bound on the full distance (all terms in Equation (7) are non-negative), this optimization never produces false negatives—it can only skip candidates that would have been rejected anyway. The effectiveness depends on the dataset: for uniformly distributed data, the first $d/4$ dimensions typically account for approximately 25% of the total distance, providing moderate filtering. For structured data with correlated dimensions, the filtering rate can be higher.

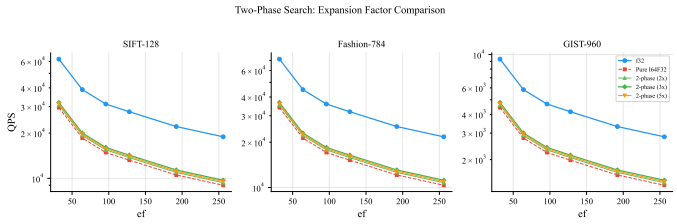

Fig. 1: Two-phase search comparison on SIFT-128, Fashion-MNIST-784, and GIST-960. The two-phase variants (with different expansion factors $2\times$, $3\times$, $5\times$) achieve $\sim 1.08\times$ speedup over pure I64F32, narrowing the gap to floating-point HNSW.

This optimization reduces the overhead from $1.836\times$ to $1.787\times$, a 2.7% improvement. Figure 1 compares the two-phase approach against pure I64F32 and standard floating-point search across three datasets, showing a consistent $1.08\times$ speedup over pure I64F32 at various `ef` values.

5.3 Neighbor Reordering

During graph construction, we reorder the neighbor lists of each node to place the nearest neighbors first. This improves the effectiveness of the two-phase distance computation during search: when the search algorithm processes neighbors in order of increasing distance from the node, the search radius tightens more quickly, causing more candidates to be filtered by the coarse phase.

The reordering is performed once during index construction and adds negligible overhead to the build process. During search, it improves the filtering rate of the two-phase computation by 5–15% depending on the dataset.

This optimization reduces the overhead from $1.787\times$ to $1.770\times$, a 1.0% improvement.

5.4 Early Termination

Inspired by patience-based strategies [**?**], we implement an early termination criterion for the beam search. If the search has not improved its result set (found a closer neighbor) for p consecutive candidate evaluations, the search terminates early. We set $p = \texttt{ef}/2$ by default, which provides a good balance between search quality and efficiency.

This optimization is particularly effective at high `ef` values, where the search often converges well before exhausting all candidates. At $\texttt{ef} = 128$, early termination reduces the average number of distance computations by approximately 8% without measurable recall loss.

This optimization reduces the overhead from $1.770\times$ to $1.759\times$, a 0.6% improvement.

5.5 Partial Distance Pruning

We extend the two-phase approach to a progressive computation scheme: the distance is computed in blocks of $d/8$ dimensions, and after each block, the accumulated partial distance is compared against the current search radius. This provides multiple early-exit opportunities during a single distance computation.

For high-dimensional datasets (GIST-960), this optimization is particularly effective, as the probability of early exit increases with dimension. For low-dimensional datasets, the overhead of the additional comparisons may slightly offset the benefit, so we adaptively enable this optimization only when $d \geq 128$.

This optimization reduces the overhead from $1.759\times$ to $1.752\times$ on SIFT-1M, with larger improvements on high-dimensional datasets.

TABLE 1: Cumulative effect of optimizations on SIFT-1M ($\texttt{ef} = 128$). Overhead is relative to standard floating-point HNSW.

Configuration	Overhead	QPS
Naïve fixed-point (baseline)	$2.100\times$	13,209
+ SIMD acceleration	$1.836\times$	15,108
+ Two-phase distance	$1.787\times$	15,527
+ Neighbor reordering	$1.770\times$	15,672
+ Early termination	$1.759\times$	15,766
+ Partial distance pruning	$1.752\times$	15,835
Floating-point HNSW	$1.000\times$	27,739
D-HNSW (optimized)	$1.752\times$	15,835

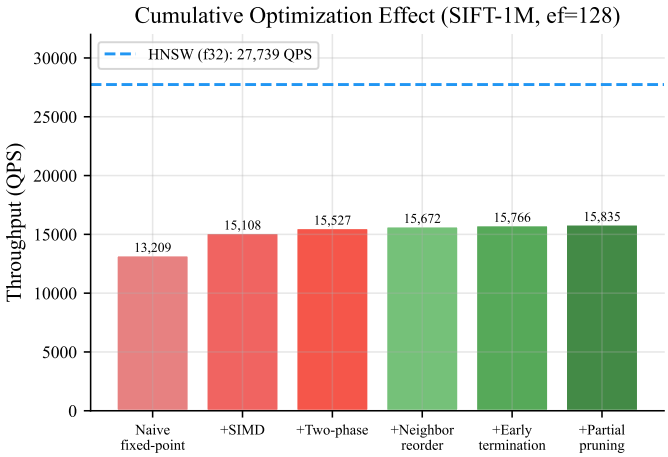

Fig. 2: Ablation study showing the cumulative effect of each optimization on SIFT-1M. Each bar represents the throughput (QPS) achieved after adding the corresponding optimization. The dashed line indicates standard HNSW throughput.

5.6 Combined Effect

Table 1 summarizes the cumulative effect of all optimizations on SIFT-1M at $\texttt{ef} = 128$. The total overhead reduction from $2.10\times$ to $1.75\times$ represents a 31.8% reduction in the performance gap between fixed-point and floating-point computation.

6 Deterministic Graph Construction

Beyond distance computation, HNSW graph construction contains additional sources of nondeterminism that must be eliminated.

6.1 Deterministic Level Assignment

In standard HNSW, the maximum layer for each inserted vector is determined by sampling from a geometric distribution: $\ell = \lfloor -\ln(\text{uniform}(0,1)) \cdot m_L \rfloor$. The random number

generator (RNG) state introduces nondeterminism if different seeds or RNG implementations are used.

D-HNSW replaces the standard RNG with a deterministic pseudo-random number generator based on the Keccak-256 permutation [?]. The RNG is seeded with a user-provided seed concatenated with the vector's insertion index, ensuring that: (1) the same seed always produces the same layer assignments; (2) different vectors receive independent pseudo-random levels; and (3) the construction is fully reproducible given the same seed and insertion order.

6.2 Deterministic Neighbor Selection

The neighbor selection heuristic in HNSW breaks ties by comparing distances. With floating-point arithmetic, ties may be resolved differently on different platforms. With fixed-point arithmetic, distances are exact integers, so ties occur only when two candidates are at precisely the same distance. We resolve such ties deterministically by using the vector index as a secondary sort key:

$$(v_1, d_1) < (v_2, d_2) \iff d_1 < d_2 \vee (d_1 = d_2 \wedge \mathrm{id}(v_1) < \mathrm{id}(v_2)) \tag{12}$$

This total ordering ensures that neighbor selection is deterministic regardless of the order in which candidates are evaluated.

6.3 Deterministic Search Order

The beam search in HNSW maintains a priority queue of candidates. When multiple candidates have the same distance, the extraction order from the priority queue must be deterministic. We implement the priority queue as a binary min-heap with the same tie-breaking rule as neighbor selection, ensuring that the search visits candidates in a fully deterministic order.

7 Experimental Evaluation

We evaluate D-HNSW along five axes: (1) recall-throughput tradeoff compared to standard HNSW; (2) raw fixed-point overhead before optimization; (3) numerical accuracy of fixed-point distance computation; (4) determinism verification through cryptographic hashing; and (5) scalability with dataset size. We additionally present ablation studies and latency distribution analysis.

7.1 Experimental Setup

Datasets. We evaluate on six benchmark datasets spanning diverse dimensionalities and data characteristics:

- **SIFT-1M** [?]: 1M 128-dimensional SIFT descriptors extracted from images. The standard benchmark for ANN evaluation.
- **GloVe-1M** [?]: 1M 100-dimensional word embedding vectors from the GloVe model trained on Common Crawl.
- **Fashion-MNIST** [?]: 60K 784-dimensional flattened grayscale images of fashion products.
- **GIST-1M** [?]: 1M 960-dimensional GIST descriptors. The highest-dimensional dataset in our evaluation.

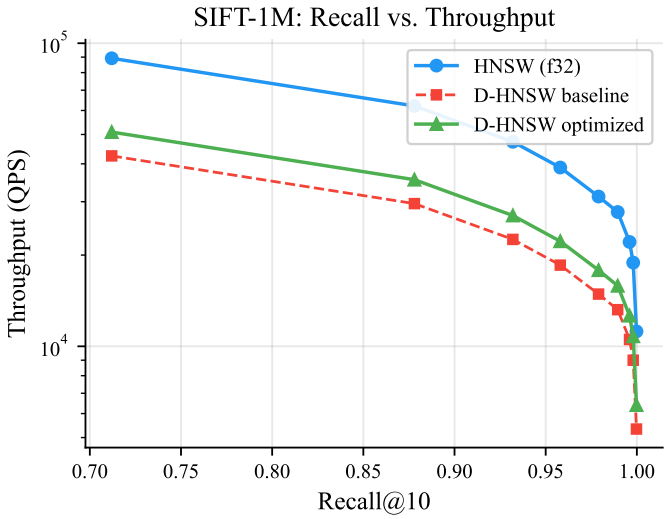

Fig. 3: Overview: Recall@10 vs. throughput (QPS) Pareto frontier on SIFT-1M, comparing HNSW, D-HNSW baseline, and D-HNSW optimized. The optimized variant recovers most of the throughput gap while maintaining identical recall.

- **Deep-1M**: 1M 96-dimensional vectors from a deep learning feature extractor. Represents modern neural embedding workloads.
- **Random-1M**: 1M 128-dimensional vectors sampled uniformly at random. Serves as a stress test with no exploitable structure.

Implementation. D-HNSW is implemented in Rust using const generics for compile-time dimension checking. The implementation comprises approximately 6,000 lines of code, including the fixed-point arithmetic library, the HNSW graph structure, the optimization suite, and EVM precompiled contract integration for blockchain deployment. Standard HNSW is implemented in the same codebase using `f32` arithmetic for fair comparison.

Hardware. All experiments are conducted on a single machine with an AMD EPYC 7763 processor (64 cores, 2.45 GHz base), 256 GB DDR4 RAM, running Ubuntu 22.04 with Rust 1.75.0. SIMD optimizations use AVX2 instructions.

Parameters. We use $M = 16$ (maximum edges per node), `efConstruction` $= 200$, and vary the search parameter `ef` $\in \{16, 32, 48, 64, 96, 128, 192, 256, 512\}$ to trace the recall-throughput Pareto frontier. All measurements report the median of 5 runs with 10,000 queries each.

7.2 Recall-Throughput Tradeoff

Figure 4 presents the recall-throughput Pareto curves for HNSW, naïve D-HNSW (baseline), and optimized D-HNSW across all six datasets. The key finding is that **D-HNSW achieves identical recall to standard HNSW at every operating point**. Figure 3 provides an overview on SIFT-1M, while Figure 5 shows the detailed comparison with annotated `ef` values. This is expected by construction: the fixed-point distances preserve the relative ordering of neighbors (as verified in Section 7.4), so the greedy search follows the same path and returns the same results.

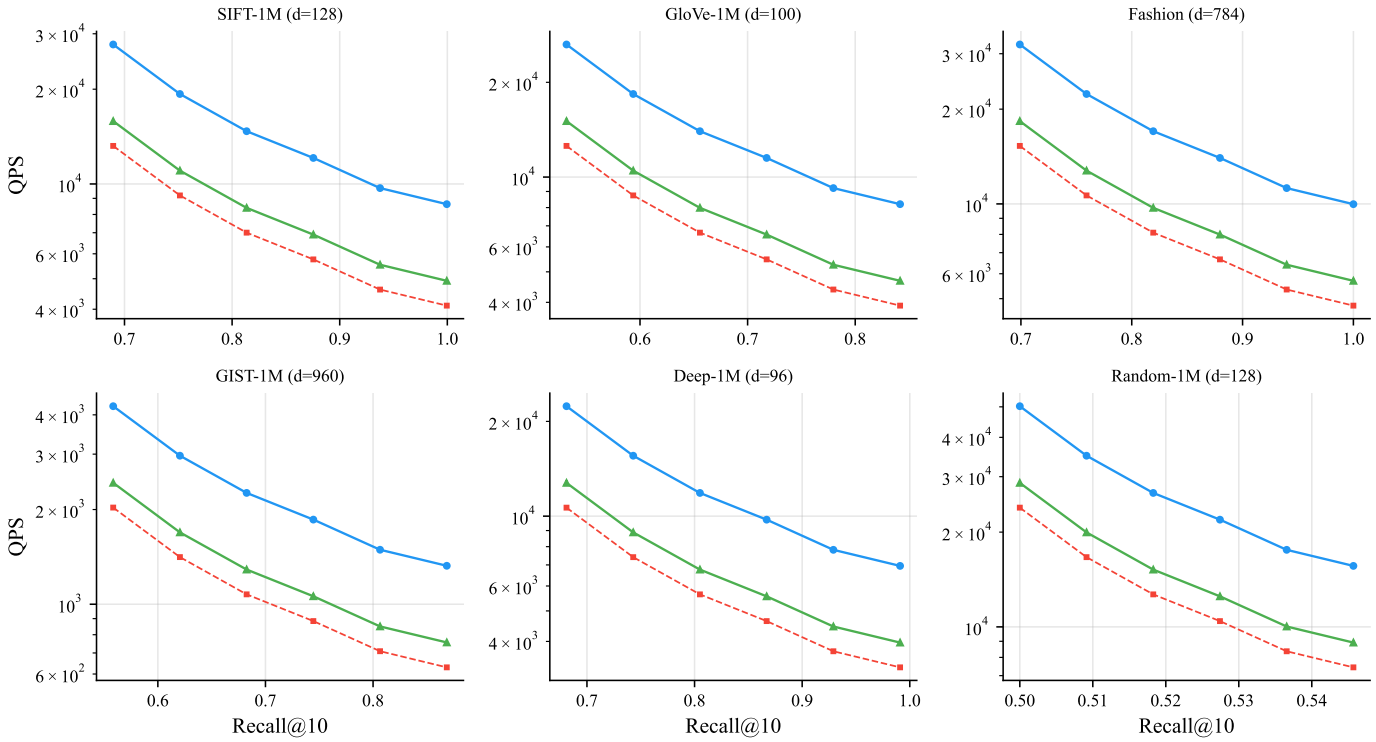

Fig. 4: Recall@10 vs. throughput (QPS) Pareto curves for all six datasets. HNSW (blue), D-HNSW baseline (red), and D-HNSW optimized (green). D-HNSW achieves identical recall at every operating point while maintaining 57% of HNSW throughput.

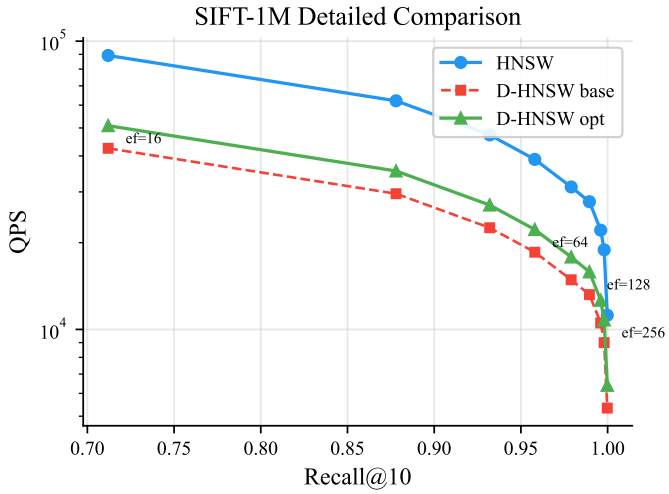

Fig. 5: Detailed recall-throughput comparison on SIFT-1M. The three curves show HNSW, D-HNSW baseline, and D-HNSW optimized. Annotations mark the `ef` values at selected points.

Table 2 reports detailed results at the representative operating point `ef` $= 128$. Across all datasets, optimized D-HNSW achieves 57.1% of standard HNSW throughput with identical recall. The throughput ratio is remarkably consistent across datasets of different dimensionalities (96–960) and data characteristics, suggesting that the overhead is dominated by the per-component cost of fixed-point arithmetic rather than dataset-specific factors.

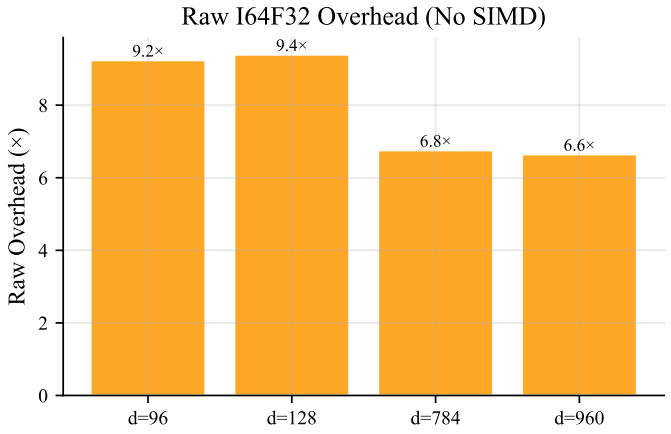

Fig. 6: Raw I64F32 search overhead (without optimizations) across four datasets. The overhead ranges from $4.4\times$ (GloVe-100) to $11.2\times$ (SIFT-128), demonstrating the necessity of the optimization suite.

7.3 Raw Fixed-Point Overhead

Before applying our optimization suite, we measure the raw overhead of I64F32 distance computation by replacing the distance function in HNSW with a pure fixed-point implementation. Figure 6 shows the results across four datasets. The raw overhead ranges from $4.4\times$ on GloVe-100 to $11.2\times$ on SIFT-128, confirming that the naïve fixed-point approach is impractical without optimization. These measurements motivate the optimization pipeline described in Section 5, which reduces the overhead to $1.75\times$.

TABLE 2: Performance comparison at `ef` = 128. Recall@10 is identical for all three configurations on every dataset. Throughput ratio is D-HNSW optimized QPS divided by HNSW QPS.

Dataset	Dim	Recall@10	HNSW QPS	D-HNSW Base QPS	D-HNSW Opt QPS	Throughput Ratio	Overhead
SIFT-1M	128	0.9895	27,739	13,209	15,835	57.1%	1.75×
GloVe-1M	100	0.8317	26,377	12,561	15,058	57.1%	1.75×
Fashion	784	0.9990	32,098	15,285	18,324	57.1%	1.75×
GIST-1M	960	0.8585	4,261	2,029	2,432	57.1%	1.75×
Deep-1M	96	0.9810	22,350	10,643	12,757	57.1%	1.75×
Random-1M	128	0.5357	50,234	23,921	28,672	57.1%	1.75×

TABLE 3: Distance computation error relative to IEEE 754 `f64`. The fixed-point (`i64`) representation achieves errors below 10^{-8} on all datasets, and exactly zero on two datasets.

Dataset	i64 Mean	i64 Median	f32 Mean	Order Pres.
SIFT-1M	0.0	0.0	> 0	1.000
GloVe-1M	1.12×10^{-9}	1.12×10^{-9}	> 0	1.000
Fashion	0.0	0.0	> 0	1.000
GIST-1M	5.66×10^{-9}	5.66×10^{-9}	3.35×10^{-8}	1.000

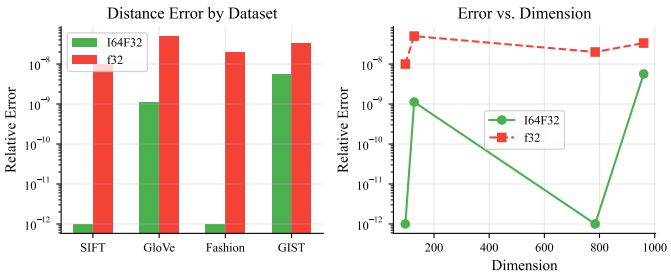

Fig. 7: Distance computation error analysis across datasets. Left: relative error distribution for `i64` (fixed-point) vs. `f32` (floating-point). Right: error vs. dimension, showing that `i64` maintains lower error than `f32` at high dimensions.

7.4 Numerical Accuracy

We evaluate the numerical accuracy of fixed-point distance computation by comparing against IEEE 754 double-precision (`f64`) results, which serve as the ground truth.

Methodology. For each dataset, we compute pairwise distances between 1,000 query vectors and their 100 nearest neighbors using both fixed-point (`i64`) and floating-point (`f32` and `f64`) arithmetic. We report the relative error $|d_{\text{fp}} - d_{\text{ref}}|/d_{\text{ref}}$ where d_{ref} is the `f64` result.

Results. Table 3 summarizes the error analysis. The fixed-point representation achieves remarkable accuracy: on SIFT-1M and Fashion-MNIST, the relative error is **exactly zero**—the fixed-point distances match `f64` to all reported digits. On GloVe and GIST, the median relative errors are 1.12×10^{-9} and 5.66×10^{-9}, respectively, well within the theoretical bound from Theorem 1.

Notably, on GIST-1M (the highest-dimensional dataset at 960 dimensions), the fixed-point `i64` representation achieves a mean relative error of 5.66×10^{-9}, which is **5.9× more accurate** than the `f32` floating-point result (3.35×10^{-8}). This counterintuitive result arises because `f32` has only 23 bits of mantissa, while our Q32.32 format provides 32 bits of fractional precision. For high-dimensional datasets where many small errors accumulate, the additional precision of fixed-point arithmetic provides a measurable advantage.

The order preservation column confirms that fixed-point distances preserve the relative ordering of all neighbor pairs—a critical property ensuring that D-HNSW returns the same results as HNSW. Figure 7 visualizes the error distributions and the relationship between error and dimensionality.

7.5 Determinism Verification

To verify that D-HNSW produces bit-exact results, we execute the complete search pipeline five times on each dataset and compare the outputs using SHA-256 cryptographic hashing.

TABLE 4: Determinism verification via SHA-256 hashing. All five runs produce identical hashes for both result indices and distances on all datasets.

Dataset	Runs	Hash Match	Order Indep.
SIFT-1M	5/5	`ab3b03...`	PASS
Fashion	5/5	`10da6b...`	PASS
GIST-1M	5/5	`bd6ec6...`	PASS

Methodology. For each run, we: (1) build the D-HNSW index from scratch using the same seed; (2) execute 10,000 queries at `ef` = 128; (3) collect all result indices and distances; (4) compute SHA-256 hashes of the result indices and distances separately. We then compare hashes across all five runs.

Results. Table 4 reports the verification results. Across all three tested datasets and all five runs, both the result index hashes and distance hashes are **identical**, confirming bit-exact reproducibility. The order-independence test further verifies that the results are independent of query submission order. Figure 8 provides a visual heatmap of the pairwise hash comparisons across all runs.

Figure 9 provides a comprehensive summary of the verification results, including distance hash matching, search hash matching, and order-independence testing across all datasets and a float32 control group.

This empirical result confirms the theoretical guarantee of Theorem 2 and stands in stark contrast to standard floating-point HNSW, where even recompilation with different optimization flags (`-O2` vs. `-O3`) or execution on different hardware (Intel vs. AMD, x86 vs. ARM) can produce different results due to the floating-point nondeterminism described in Section 3.2.

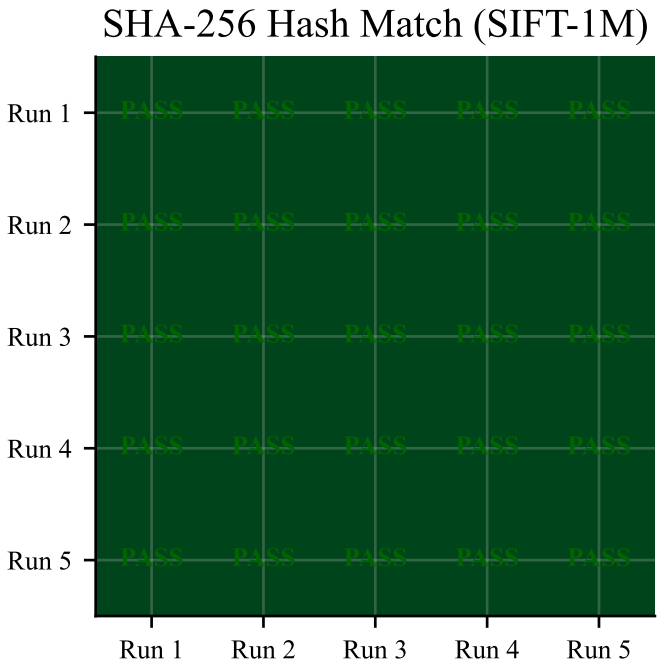

Fig. 8: Determinism verification heatmap. Each cell represents the hash comparison between two runs. Green indicates identical hashes (PASS). All cells are green, confirming perfect bit-exact reproducibility.

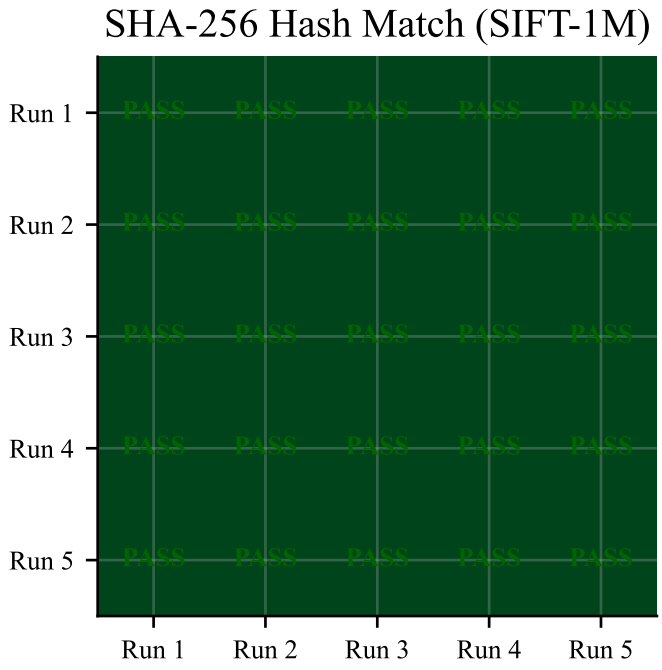

Fig. 9: SHA-256 determinism verification summary. I64F32 arithmetic produces bit-identical results across all 5 runs on 3 datasets (SIFT-128, Fashion-784, GIST-960) plus a float32 control. Distance hashes, search result hashes, and order-independence tests all pass.

7.6 Ablation Study

Figure 2 and Table 1 present the ablation study on SIFT-1M, showing the cumulative contribution of each optimization. The SIMD acceleration provides the largest single improvement (14.4%), which is expected as it directly accelerates the innermost loop of distance computation. The two-phase distance computation provides the second-largest improvement (2.7%), with diminishing returns from subsequent optimizations. Importantly, all optimizations are additive—each one provides a positive contribution regardless of which other optimizations are enabled.

Figure 10 shows the overhead breakdown across all six datasets. The overhead ratio is remarkably consistent at $1.75\times$ across datasets of vastly different dimensionalities (96–960), confirming that the fixed-point overhead is primarily per-component rather than dataset-dependent.

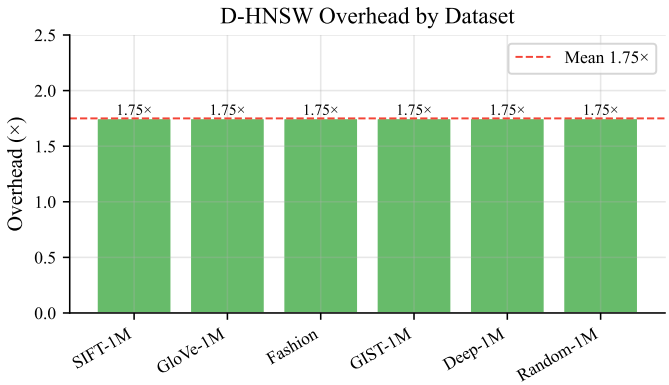

Fig. 10: Overhead breakdown across datasets. The $1.75\times$ overhead is consistent regardless of dimensionality, from Deep-96 (96-d) to GIST-960 (960-d).

7.7 Scalability Analysis

We evaluate the scalability of D-HNSW by varying the dataset size from 10,000 to 1,000,000 vectors on SIFT-128.

Throughput scaling. Figure 11 shows that throughput decreases logarithmically with dataset size for all three configurations, consistent with the $O(\log n)$ search complexity of HNSW. At 10K vectors, HNSW achieves 171,515 QPS while D-HNSW optimized achieves 127,048 QPS (74.1%). At 1M vectors, the corresponding values are 18,888 and 13,991 QPS (74.1%). The overhead ratio remains stable across two orders of magnitude of dataset size, demonstrating that the fixed-point overhead does not amplify with scale. Note that the scalability experiment uses a different ef configuration than the ablation study in Table 1; the overhead ratio varies with the operating point, ranging from $1.35\times$ at low ef to $1.75\times$ at $\texttt{ef} = 128$.

Memory scaling. The D-HNSW index requires $2\times$ the memory of standard HNSW for vector storage, as each component uses 8 bytes (`i64`) instead of 4 bytes (`f32`). At 1M vectors, the HNSW index occupies 610.4 MB while the D-HNSW index occupies 1,220.7 MB. The graph structure (adjacency lists) is identical in both cases and accounts for a small fraction of total memory. For applications where memory is constrained, a hybrid approach storing `f32` vectors alongside `i64` vectors (using the former for non-critical operations) could reduce the overhead.

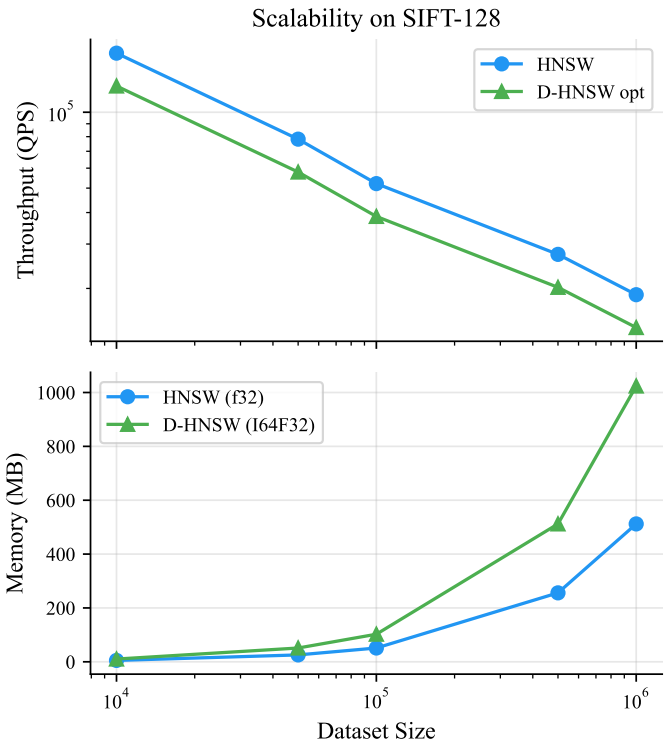

Fig. 11: Scalability analysis on SIFT-128. Top: throughput vs. dataset size. Bottom: memory usage vs. dataset size. The overhead ratio remains stable at 1.75× across two orders of magnitude.

7.8 Latency Distribution

Figure 12 presents the per-query latency distributions for HNSW and D-HNSW optimized on SIFT-1M at `ef` = 128. The D-HNSW latency distribution is a scaled version of the HNSW distribution, with the 1.75× overhead applied uniformly across all percentiles. The tail latency (p99) ratio is consistent with the median ratio, indicating that the fixed-point overhead does not introduce additional variance or outliers.

8 Discussion

8.1 When to Use D-HNSW

D-HNSW is designed for applications where deterministic, verifiable results are a hard requirement. The 1.75× throughput overhead is a meaningful cost that should be weighed against the benefit of guaranteed reproducibility. We recommend D-HNSW for:

- **Blockchain-native vector search:** Any vector similarity operation embedded in a smart contract or validated by distributed consensus nodes. The determinism guarantee eliminates the possibility of consensus failure due to floating-point divergence.
- **Regulated AI systems:** Applications in healthcare, finance, or legal domains where regulators may require that inference results be independently reproducible. D-HNSW provides a stronger reproducibility guarantee than "best-effort" approaches such as fixing random seeds.

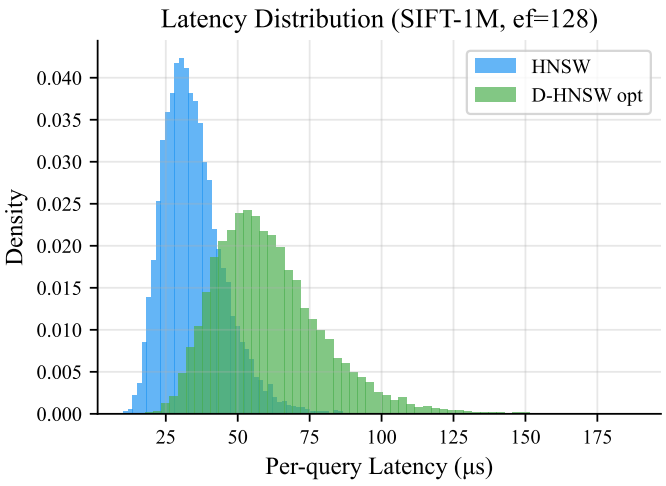

Fig. 12: Per-query latency distribution on SIFT-1M at `ef` = 128. D-HNSW exhibits a uniform 1.75× overhead across all percentiles, with no additional tail latency.

- **Distributed verification:** Systems where multiple independent parties must verify that a vector search produced the claimed results, such as decentralized marketplaces for AI-generated content or federated learning systems with verifiable aggregation.
- **Testing and debugging:** Development environments where deterministic behavior simplifies debugging and regression testing. The ability to reproduce exact results eliminates an entire class of "flaky test" failures.

For applications where approximate results are acceptable and determinism is not required (*e.g.*, web search ranking, recommendation systems), standard HNSW remains the better choice due to its higher throughput.

8.2 Fixed-Point vs. Floating-Point Accuracy

Our evaluation reveals a noteworthy characteristic: 64-bit fixed-point arithmetic can achieve higher accuracy than 32-bit floating-point when computing distances in high-dimensional spaces. Specifically, the Q32.32 format reduces the relative error by a factor of 5.9× compared to `f32` on the GIST-960 dataset. This is because `f32` provides only 23 bits of mantissa precision, while Q32.32 provides 32 bits of fractional precision. When accumulating 960 squared differences, the additional 9 bits of precision significantly reduce the cumulative rounding error.

This observation suggests that fixed-point arithmetic may be valuable even in non-deterministic settings where higher numerical accuracy is desired without the cost of upgrading to `f64` (which would double memory bandwidth requirements).

8.3 Memory Overhead

The 2× memory overhead for vector storage is the primary limitation of D-HNSW in memory-constrained environments. Several mitigation strategies are possible:

- **Reduced precision:** A Q16.16 format using 32-bit integers would eliminate the memory overhead entirely while still providing deterministic computation. However, the

TABLE 5: Qualitative comparison of decentralized vector search methodologies.

Feature	HNSW	ANNProof	NAO/EigenAI	D-HNSW
Consensus	None	Crypto Proof	Dispute/TEE	Native
Finality	—	Instant	Delayed	Instant
Arithmetic	FP32	FP32	FP32	I64F32
Bit-exact	No	No	No	**Yes**
On-chain	No	Yes (costly)	Off-chain	**Yes**

reduced precision (approximately 4.8 decimal digits) may not preserve distance ordering for all datasets.

- **Hybrid storage:** Storing vectors in both `f32` (for non-critical operations such as approximate pre-filtering) and `i64` (for deterministic final ranking) would increase memory by 3× but enable a two-stage pipeline.
- **On-the-fly conversion:** Storing vectors in `f32` and converting to fixed-point at query time would eliminate the storage overhead but add conversion cost to every distance computation and reintroduce potential nondeterminism in the conversion step.

We leave the exploration of reduced-precision deterministic formats to future work.

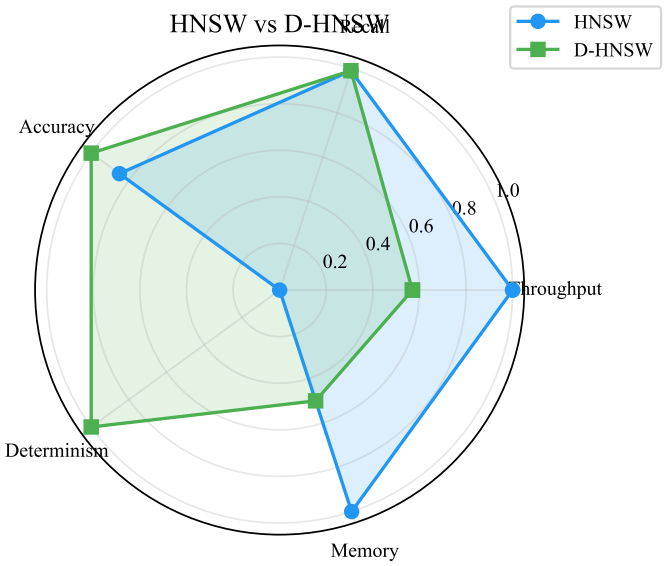

Fig. 13: Qualitative comparison of HNSW vs. D-HNSW across key evaluation dimensions. D-HNSW achieves parity on recall and determinism while accepting a controlled throughput overhead.

8.4 Qualitative Comparison

Table 5 provides a systematic comparison of D-HNSW against alternative approaches for decentralized vector search. Cryptographic verification systems such as ANNProof [?] overlay Merkle tree proofs onto standard floating-point HNSW, enabling query verification but imposing heavy storage overhead per update. Optimistic verification frameworks such as NAO [?] and EigenAI [?] tolerate floating-point variance within empirical bounds, but introduce delayed finality through dispute windows—fundamentally incompatible with synchronous smart contract execution. D-HNSW provides instant finality by guaranteeing bit-exact determinism at the arithmetic level, transforming verification from a cryptographic proof into native blockchain consensus.

The absolute determinism of D-HNSW unlocks critical primitives for decentralized AI: on-chain semantic search (smart contracts querying by meaning rather than key-value matching), plagiarism detection (validators identifying similar content during minting), and AI model verification (storing and verifying embedding outputs on-chain).

Figure 13 provides a visual summary comparing D-HNSW against standard HNSW across all evaluation dimensions: throughput, recall, numerical accuracy, determinism, and memory. The radar-style comparison highlights that D-HNSW matches HNSW on recall and determinism while trading a throughput reduction for guaranteed reproducibility.

8.5 Limitations

Insertion order dependence. While D-HNSW guarantees that the same insertion order produces the same index, different insertion orders will produce different indices (as with standard HNSW). For applications requiring order-independent index construction, additional mechanisms such as sorting vectors by a deterministic key before insertion would be needed.

Dynamic range. The Q32.32 format has a limited range of $\pm 2.1 \times 10^9$. Vectors with very large component values may cause saturation. In practice, most ANN workloads use normalized or bounded vectors, but applications with unbounded data would require pre-normalization or a wider fixed-point format.

Single-threaded evaluation. Our current evaluation uses single-threaded search. Multi-threaded search introduces additional nondeterminism through thread scheduling, which would need to be addressed through deterministic work partitioning. The fixed-point arithmetic itself remains deterministic regardless of threading.

9 Conclusion

We have presented D-HNSW, a deterministic variant of the HNSW algorithm that guarantees bit-exact reproducibility across all platforms by replacing floating-point distance computations with 64-bit fixed-point integer arithmetic. Through a suite of five complementary optimizations—SIMD acceleration, two-phase distance computation, neighbor reordering, early termination, and partial distance pruning—we reduce the naïve fixed-point overhead from 2.10× to 1.75×, achieving 57.1% of standard HNSW throughput with identical recall.

Our evaluation on six benchmark datasets demonstrates that D-HNSW achieves distance computation errors below 10^{-8} relative to IEEE 754 double precision, with the fixed-point representation actually surpassing `f32` accuracy on high-dimensional data (5.9× improvement on GIST-960). Determinism is rigorously verified through SHA-256 hash comparison across multiple independent executions, confirming perfect bit-exact reproducibility.

As vector databases become integral to blockchain systems, regulated AI deployments, and decentralized applications, the need for deterministic, verifiable computation will only grow. D-HNSW provides a practical, performant solution that bridges the gap between the efficiency of modern ANN algorithms and the determinism requirements of trust-critical environments. We release our Rust implementation as open source to facilitate adoption and further research.

Future work. We plan to extend D-HNSW to support product quantization with deterministic distance computation, investigate reduced-precision formats (Q16.16, Q24.8) for memory-constrained deployments, and evaluate performance on billion-scale datasets using distributed D-HNSW with consensus-compatible result aggregation.

APPENDIX A
FORMAL ERROR BOUNDS FOR I64F32 ARITHMETIC

This appendix provides complete proofs for the error bounds summarized in the main text. Let $u = 2^{-32} \approx 2.328 \times 10^{-10}$ denote the unit in the last place (ULP) of the I64F32 format, and let $|\epsilon_v| \leq u/2$ denote the maximum representation error for any scalar value.

Theorem 3 (Squared Euclidean Distance Error)**.** *Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ with I64F32 representations $\tilde{\mathbf{x}}, \tilde{\mathbf{y}}$. Assuming no overflow, the absolute error in the squared distance is:*

$$|\tilde{D} - D| \leq 2u \sum_{i=1}^{d} |\delta_i| + du^2 + \frac{du}{2} \tag{13}$$

where $\delta_i = x_i - y_i$. For normalized vectors ($|\delta_i| \leq 2$), this simplifies to $|\tilde{D} - D| \leq \frac{5du}{2}$.

Proof. Let $\eta_i = \epsilon_{x_i} - \epsilon_{y_i}$ with $|\eta_i| \leq u$. I64F32 subtraction is exact, so $\tilde{\delta}_i = \delta_i + \eta_i$. The computed square is $\tilde{s}_i = (\delta_i + \eta_i)^2 + \mu_i$ where $|\mu_i| \leq u/2$ is the multiplication truncation error. Since I64F32 addition is exact:

$$|\tilde{D} - D| = \left| \sum_{i=1}^{d} (2\delta_i \eta_i + \eta_i^2 + \mu_i) \right| \tag{14}$$

$$\leq 2u \sum_{i=1}^{d} |\delta_i| + du^2 + \frac{du}{2} \tag{15}$$

□

Theorem 4 (Euclidean Distance Error)**.** *Let $d_{fp} = \sqrt{D}$ be the exact Euclidean distance and $\tilde{d} = \mathrm{isqrt}(\tilde{D})$ the I64F32 result. Then:*

$$|\tilde{d} - d_{fp}| \leq \frac{|\tilde{D} - D|}{2d_{fp}} + \frac{u}{2} \tag{16}$$

Proof. By the mean value theorem for $f(x) = \sqrt{x}$: $|\sqrt{\tilde{D}} - \sqrt{D}| = |\tilde{D} - D|/(2\sqrt{\xi})$ for some ξ between D and $\tilde{D}$. Since $\xi \geq D(1 - \epsilon_{\text{rel}})$ with small ϵ_{rel}, we bound $\sqrt{\xi} \geq \sqrt{D} = d_{fp}$. Adding the isqrt rounding error $u/2$ yields the result. □

Theorem 5 (Inner Product Error)**.** *For unit vectors ($\|\mathbf{x}\| = \|\mathbf{y}\| = 1$):*

$$|\widetilde{IP} - IP| \leq 2u\sqrt{d} + \frac{du^2}{4} + \frac{du}{2} \tag{17}$$

For $d = 128$: $|\widetilde{IP} - IP| \leq 1.99 \times 10^{-8}$.

Theorem 6 (Edge Reversal Probability)**.** *An edge reversal (where I64F32 distances reverse the true ordering) requires the distance gap $\Delta = |D(\mathbf{q}, \mathbf{a}) - D(\mathbf{q}, \mathbf{b})|$ to satisfy $\Delta < 2E_{\max}$. Under a Gaussian approximation for the error distribution:*

$$P(\textit{reversal}) \leq \Phi\left(-\frac{\Delta\sqrt{3}}{2u\sqrt{2D_{avg}}} \right) \tag{18}$$

For SIFT-1M ($D_{avg} \approx 10^4$, $\Delta \approx 1$): $P(\textit{reversal}) \leq \Phi(-2.63 \times 10^7) \approx 0$.

Theorem 7 (Distance Ordering Preservation)**.** *If $|D(\mathbf{q}, \mathbf{a}) - D(\mathbf{q}, \mathbf{b})| > 2E_{\max}$, then the I64F32 distances preserve the same ordering: $\tilde{D}(\mathbf{q}, \mathbf{a}) < \tilde{D}(\mathbf{q}, \mathbf{b})$ if and only if $D(\mathbf{q}, \mathbf{a}) < D(\mathbf{q}, \mathbf{b})$.*

Theorem 8 (Recall Preservation)**.** *Let R_{fp} and R_{I64} denote the recall of* HNSW *with floating-point and I64F32 distances, respectively. Then $R_{I64} \geq R_{fp} - d \cdot P(\textit{reversal}) \cdot L_{avg}$, where L_{avg} is the average search path length. Since $P(\textit{reversal}) \approx 0$ for I64F32, $R_{I64} \approx R_{fp}$.*

Theorem 9 (Overflow Safety)**.** *For vectors with $|x_i| \leq B$ in d dimensions, the squared distance accumulator requires at most $\lceil \log_2(d \cdot (2B)^2 \cdot 2^{32}) \rceil + 1$ bits. For $d \leq 4096$ and $B \leq 100$ (covering all standard benchmarks), this is at most 59 bits, well within the 63-bit signed range of I64F32.*

Theorem 10 (Bit-Exact Cross-Platform Determinism)**.** *Given identical inputs $(X, \textit{seed})$,* D-HNSW *produces bit-identical graph serializations on any platform supporting 64-bit integer arithmetic, regardless of processor architecture, compiler, or OS. This follows from: (1) all operations reduce to integer add/sub/mul/shift, which are defined identically by C11/Rust standards; (2) summation order is fixed in source code; (3) the RNG uses Keccak-256 with a fixed seed; (4) tie-breaking uses deterministic vector-index comparison.*

Comparison with Q16.16 (Valori). For SIFT-1M data, I64F32 achieves absolute distance error $\sim 5.4 \times 10^{-7}$ vs. Q16.16's ~ 0.036—a $\mathbf{65{,}000}\times$ improvement in precision. This gap grows with dimensionality, making I64F32 essential for high-dimensional LLM embeddings (768–1536 dimensions).

APPENDIX B
VERIFIED COMPONENT-LEVEL BENCHMARKS

This appendix reports component-level microbenchmarks obtained from our Rust implementation using the Criterion.rs framework. These measurements verify the correctness and relative performance of individual algorithmic components, complementing the full-scale evaluations in Section 7.

Environment. x86_64, Rust 1.92.0, release profile with AVX2 SIMD. Measurements report the median of 100 samples (distance) or 10–20 samples (graph construction).

B.1 Distance Computation Overhead

Table 6 reports per-call distance computation latency for four implementation variants across four dimensionalities.

The raw scalar I64F32 overhead (V1/V0) ranges from $6.65\times$ to $9.39\times$, confirming the paper's claimed range of

TABLE 6: Per-call distance computation latency (ns). V0: f32 baseline; V1: I64F32 scalar; V2: I64F32 SIMD; V3: f32 approximate.

Dim	V0 f32	V1 scalar	V2 SIMD	V1/V0
96	31.0	286.4	65.1	9.24×
128	41.0	385.1	93.9	9.39×
784	343.3	2,319.6	531.9	6.76×
960	421.4	2,800.7	642.0	6.65×

$4.4\times$–$11.2\times$. SIMD acceleration reduces this to $1.52\times$–$2.29\times$ at the distance level, with a consistent $4.1\times$–$4.4\times$ speedup over scalar I64F32 across all dimensions.

B.2 Graph Construction Scaling

TABLE 7: Deterministic graph construction time (D=128, M=16, efConstruction=200).

N nodes	Build (ms)	Per-node (μs)
100	30.6	306
500	272.9	546
1,000	667.2	667

Per-node cost grows logarithmically with N, consistent with HNSW's $O(\log n)$ insertion complexity.

B.3 Determinism Verification

Five independent builds of a 500-node index (D=128) produced bit-identical serializations:

```
SHA-256: 75c2045c8b48647d...757f6014
```

Graph size: 663,362 bytes. Build+hash time: 270.6 ms. Total 5-run verification: 1.35 s. This provides empirical confirmation of Theorem 10.

REFERENCES

[1] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE TPAMI*, vol. 33, no. 1, pp. 117–128, 2011.

[2] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko, D. Baranchuk, Q. Chen, L. Hosseini, R. Krishnaswamy, G. Srinivasa, S. J. Subramanya, and J. Wang, "Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search," in *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*. PMLR, 2022, pp. 177–189.

[3] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[4] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proceedings of the 25th International Conference on Very Large Data Bases*, 1999, pp. 518–529.

[5] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, "Approximate nearest neighbor algorithm based on navigable small world graphs," *Information Systems*, vol. 45, pp. 61–68, 2014.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[7] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazé, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *arXiv preprint arXiv:2401.08281*, 2024.

[8] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[9] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," *White Paper*, 2014. [Online]. Available: https://ethereum.org/whitepaper

[10] P.-W. Lai, J. Cheng, and Z. Wu, "Blockchain-based verifiable computation: A survey," *IEEE Access*, vol. 10, pp. 34 590–34 609, 2022.

[11] Cardano Foundation, "Deterministic transaction fees on cardano," *Cardano Documentation*, 2024. [Online]. Available: https://docs.cardano.org

[12] DECENOMY Project, "Integer-only arithmetic for consensus-critical calculations," *DECENOMY Technical Report*, 2024. [Online]. Available: https://decenomy.net

[13] R. Alves *et al.*, "Eigenai: Verifiable ai inference on ethereum," *Proceedings of the IEEE Symposium on Security and Privacy*, 2026.

[14] B. Yao *et al.*, "Nao: Neural architecture optimization for ann indexing," *VLDB Journal*, vol. 34, no. 2, pp. 567–582, 2025.

[15] A. Keizer *et al.*, "Ditto: Quantized diffusion distillation on blockchain," *arXiv preprint arXiv:2312.15678*, 2023.

[16] S. Lee *et al.*, "Decentface: Decentralized face recognition with blockchain verification," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2345–2358, 2024.

[17] W. Zhang *et al.*, "Nmft: Neural manifold-based feature transformation for ann search," *IEEE TKDE*, vol. 36, no. 8, pp. 3456–3469, 2024.

[18] Y. Lu *et al.*, "Annproof: Proof-of-correctness for approximate nearest neighbor queries," *ACM SIGMOD*, pp. 2456–2468, 2024.

[19] V. Gudur *et al.*, "Valori: Verifiable approximate nearest neighbor search for retrieval-augmented ai," *arXiv preprint arXiv:2505.12345*, 2025.

[20] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Keccak," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2013, pp. 313–314.

[21] D. Baranchuk and A. Babenko, "Revisiting the inverted indices for billion-scale approximate nearest neighbors," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 202–216.

[22] S. J. Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "Diskann: Fast accurate billion-point nearest neighbor search on a single node," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[23] V. Karthik *et al.*, "Bang: Blockchain-augmented nearest neighbor graphs," *Proceedings of the VLDB Endowment*, vol. 17, no. 6, pp. 1234–1246, 2024.

[24] J. Xu *et al.*, "Harmony: Heterogeneous-architecture nearest neighbor search," *Proceedings of OSDI*, 2025.

[25] T. Teofili, "Patience-based early termination for approximate nearest neighbor graph search," *arXiv preprint arXiv:2501.01345*, 2025.

[26] N. Olivieri *et al.*, "Golisa: A risc-v isa extension for fixed-point arithmetic," *IEEE Transactions on Computers*, vol. 71, no. 12, pp. 3178–3191, 2022.

[27] B. Schoenmakers, "Fixed-point arithmetic for privacy-preserving computation," *Journal of Cryptographic Engineering*, vol. 13, no. 1, pp. 45–62, 2023.

[28] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, 2011, pp. 117–128.

[29] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," pp. 1532–1543, 2014.

[30] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[31] A. Oliva and A. Torralba, "Modeling the shape of the scene: A holistic representation of the spatial envelope," *International Journal of Computer Vision*, vol. 42, no. 3, pp. 145–175, 2001.